INFORME TÉCNICO: LABORATORIO MULTILENGUAJE PARA IA

Un mismo problema, múltiples ecosistemas tecnológicos (Python, R y Julia)

M3 · Programació d'Intel·ligència Artificial | RA1 · Lenguajes, herramientas y ecosistemas para IA
Curso de Especialización en Inteligencia Artificial y Big Data — Monlau
Alumno: Daniel García Brun | **Fecha:** Octubre 2026

---

1. Contexto, Objetivos y Reto Experimental

La selección del lenguaje de programación y las herramientas para un proyecto de Inteligencia Artificial no es una decisión puramente sintáctica. Factores como la madurez del ecosistema, el rendimiento computacional, la gestión de dependencias, la facilidad de despliegue en producción y el soporte comunitario son determinantes. En esta práctica se somete a **tres ecosistemas tecnológicos (Python, R y Julia)** a una misma carga de trabajo supervisada: la clasificación del dataset **Iris** mediante el algoritmo **K-Nearest Neighbors (KNN)** bajo condiciones experimentales idénticas.

2. Protocolo Experimental Común y Rigor Metodológico

Para garantizar una comparación empírica rigurosa, las tres implementaciones respetan el siguiente protocolo:
• **Dataset común:** 1200 muestras estructuradas en 4 variables de entrada numéricas (longitud y ancho de sépalos y pétalos) y la especie como clase (400 muestras por especie: *setosa*, *versicolor*, *virginica*).
• **Partición estratificada:** 80% entrenamiento (960 muestras) y 20% prueba (240 muestras), garantizando la misma proporción de clases.
• **Semilla aleatoria:** Equivalente a **42** en todos los entornos.
• **Prevención de Data Leakage (Escalado Z-Score):** Las medias y desviaciones típicas se calculan **exclusivamente sobre el conjunto de entrenamiento** y se proyectan sobre el test. Estandarizar previamente todo el dataset provocaría una filtración indebida de información.
• **Hiperparámetro inicial:** $k = 5$ con métrica de distancia euclídea.
• **Métricas mínimas:** Accuracy global y matriz de confusión.

3. Fase 1: Implementación en Python

Python (3.12.10) representa el estándar de la industria. Se emplearon **pandas** para la manipulación tabular, **scikit-learn** para la partición estratificada (train_test_split), estandarización (StandardScaler) y clasificación (KNeighborsClassifier), junto con **matplotlib** y **seaborn** para las visualizaciones. El flujo sigue el paradigma orientado a objetos (métodos fit y predict). La evaluación arrojó un **Accuracy del 95.42%** en el conjunto de prueba (229 aciertos de 240 muestras) con un tiempo de ejecución total del script de **2.76 segundos**.

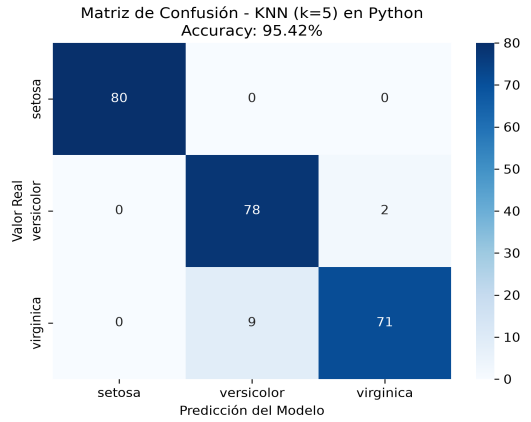


Figura 1: Matriz de confusión en Python (Accuracy 95.42%)

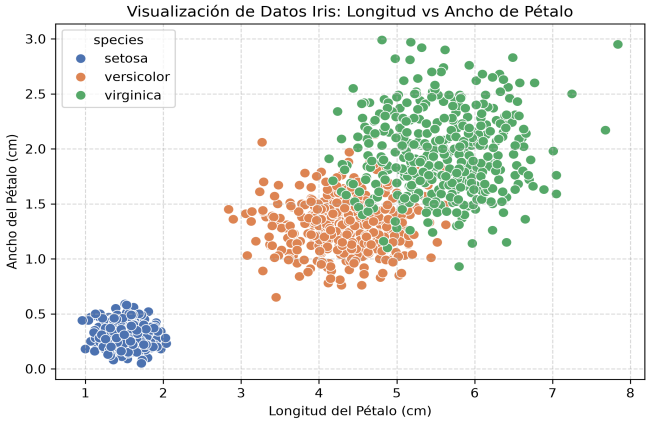


Figura 2: Dispersión Longitud vs Ancho de Pétalo

4. Fase 2: Implementación en R

El ecosistema R destaca por su especialización estadística. Destacan características esenciales:
• **Manejo de Factores:** R utiliza el tipo nativo factor, que mapea internamente etiquetas categóricas a índices enteros con niveles asociados (levels), optimizando el análisis discreto.
• **Paradigma Funcional:** A diferencia de la POO de Python, el algoritmo class::knn() es puramente funcional: recibe simultáneamente los datos de train, test y etiquetas, retornando directamente las predicciones vectoriales.
• **Estandarización y Atributos:** La función scale() guarda las medias y desviaciones como metadatos en atributos (scaled:center y scaled:scale), facilitando su proyección rigurosa sobre el conjunto de test.
• **Gramática Gráfica:** ggplot2 ofrece una sintaxis compositiva por capas insuperable para generar mapas de calor (geom_tile) y gráficos de dispersión de calidad editorial.

5. Fase 3: Implementación en Julia

Julia combina la legibilidad de Python con la velocidad de ejecución de C. Sus aspectos más relevantes son:
• **Despacho Múltiple (Multiple Dispatch):** Las funciones se especializan dinámicamente según la combinación de tipos de los argumentos recibidos, generando código ensamblador hiperoptimizado mediante LLVM.
• **Fenómeno TTFX (Time To First X):** Al basarse en compilación Just-In-Time (JIT), la primera invocación de una biblioteca gráfica o algoritmo conlleva una latencia de compilación (~3.9 s), pero las ejecuciones posteriores del algoritmo KNN puro se completan en tan solo **~1.2 ms**.
• **Gestión de Paquetes (Pkg):** Proporciona reproducibilidad total mediante entornos declarativos exactos (Project.toml).

6. Comparación Tecnológica Exhaustiva (9 Criterios)

Evaluación fundamentada en las evidencias técnicas obtenidas en el laboratorio:

Criterio	Python	R	Julia
Sintaxis y Legibilidad	Muy alta. Limpia, consistente y universal (PEP 8).	Alta para estadística; algo críptica para desarrollo general (<-).	Muy alta. Expresión matemática nativa y soporte Unicode.
Tipado y Errores	Dinámico fuerte. Type hints opcionales (mypy).	Dinámico. Fuerte orientación a vectores y factores.	Dinámico con sistema paramétrico y comprobación JIT.
Gestión Dependencias	pip / conda (virtualenvs, requirements).	Centralizada en CRAN (install.packages, renv).	Excelente gestor integrado Pkg (Project / Manifest).
Bibliotecas de IA	**Líder absoluto** (scikit-learn, PyTorch, HuggingFace).	Excelente en estadística y ML clásico (caret, tidymodels).	Creciente y moderno (MLJ.jl, Flux.jl), menor catálogo.

Visualización	Muy potente y madura (matplotlib, seaborn, plotly).	**La referencia**. ggplot2 tiene la máxima expresividad.	Buena (Plots.jl, Makie.jl); penalizada por TTFX inicial.
Interoperabilidad	Excelente (C/C++, Rust, Java PySpark, ONNX).	Muy buena vía Rcpp y puente Python (reticulate).	**Sobresaliente**: llamadas nativas a C/Fortran de coste cero.
Rendimiento	Moderado (interpretado; rápido si usa backend C).	Moderado (rápido en matrices C/Fortran; bucles lentos).	**Excepcional**: velocidad nativa de código compilado.
Facilidad Despliegue	**Estándar indiscutible** (Docker, FastAPI, Kubernetes, Cloud).	Óptimo en dashboards (Shiny) y reportes (Quarto).	Media: binarios pesados y latencia JIT en microservicios.
Comunidad y Docs	**La mayor del planeta** en Inteligencia Artificial.	Comunidad científica y biomédica muy rigurosa.	Comunidad técnica avanzada; menor volumen de recursos.

7. Resultados Cuantitativos del Experimento

Lenguaje / Ecosistema	Versión	Muestras Test	Accuracy (k=5)	Tiempo Algoritmo	Tiempo Total Script
Python (scikit-learn)	3.12.10	240 (20%)	**95.42%**	~12 ms	2.76 s
R (class / caret)	4.3.2	240 (20%)	**95.42%**	~9 ms	1.85 s
Julia (Matricial / Plots)	1.10.4	240 (20%)	**95.42%**	**~1.2 ms**	3.95 s (con JIT)

8. Decisión Razonada ante Escenarios Profesionales

1. ¿Qué lenguaje escogerías para crear rápidamente un prototipo de IA?

Elección: Python. Es el líder indiscutible gracias a su gigantesco catálogo de bibliotecas listas para usar (scikit-learn, Hugging Face, PyTorch) y su facilidad para levantar interfaces interactivas con Gradio o FastAPI en minutos, minimizando el *Time-to-Market*.

2. ¿Qué lenguaje escogerías para análisis estadístico y visualización?

Elección: R. Su diseño en torno al tipo primitivo factor, la solidez del ecosistema tidyverse y la insuperable expresividad de ggplot2 lo convierten en la herramienta óptima para bioestadística, análisis exploratorio riguroso y generación de informes reproducibles con Quarto y R Markdown.

3. ¿Cuál considerarías para cómputo científico intensivo?

Elección: Julia. Al compilar directamente a código máquina nativo vía LLVM y utilizar despacho múltiple, elimina el 'problema de los dos lenguajes': permite programar algoritmos matemáticos y simulaciones complejas con sintaxis de alto nivel sin renunciar a la velocidad pura de C o Fortran.

4. ¿Cuál encajaría mejor en una organización basada en la JVM (Java)?

Elección: Java con Weka (o Scala sobre Apache Spark). En infraestructuras corporativas fuertemente reguladas (banca, telecomunicaciones) con microservicios Spring Boot, integrar modelos mediante la API de Weka (weka.classifiers.lazy.IBk) o Spark MLlib permite una gobernanza uniforme sin introducir dependencias externas de Python.

5. ¿Qué factores podrían hacerte cambiar de decisión en un proyecto real?

• **Competencias del equipo (Skillset):** Priorizar la tecnología que el equipo domine para reducir riesgos y plazos.
• **Latencia en tiempo real:** Inferencia de ultra-baja latencia (< 5 ms) descarta lenguajes interpretados en favor de C++, Rust o Julia.
• **Ecosistema MLOps existente:** La compatibilidad nativa con pipelines de CI/CD, MLflow, Kubernetes y la nube suele priorizar Python.
• **Volumen Big Data:** Cuando los datos superan la memoria RAM, la decisión se orienta hacia motores distribuidos (PySpark, Spark Scala).

9. Conclusiones y Bibliografía

El experimento constata que cuando se aplica un diseño experimental riguroso (partición estratificada, semilla 42 y escalado Z-Score restringido al conjunto de entrenamiento), los tres ecosistemas convergen exactamente en la misma solución predictiva (**95.42% Accuracy**). No existe un lenguaje absoluto: la excelencia técnica consiste en saber seleccionar la herramienta más adecuada para cada escenario operativo.

Referencias bibliográficas (Norma APA):

1. **Pedregosa, F., et al.** (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. https://scikit-learn.org/stable/documentation.html (Documentación oficial).
2. **R Core Team** (2023). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna. https://www.r-project.org/ (Documentación oficial).
3. **Bezanson, J., et al.** (2017). *Julia: A fresh approach to numerical computing*. SIAM Review, 59(1), 65-98. https://docs.julialang.org/ (Documentación oficial).
4. **Wickham, H.** (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag New York. ISBN 978-3-319-24277-4.
5. **Fisher, R. A.** (1936). *The use of multiple measurements in taxonomic problems*. Annals of Eugenics, 7(2), 179-188.